RetinAnalysis — New Machine Setup Guide

Everything to download and install to run this repo on Windows or Mac

August 2026

Contents

Before You Start

This repo (`retinanalysis`) is a heavily customized fork — the analysis code, demo notebooks, and several utility files have significant lab-specific changes beyond the original upstream project. None of that requires any extra setup step beyond what's below: everything custom lives inside this one repository (plus one required submodule), so a normal `git clone` of the up-to-date repo already includes all of it.

This guide clones from `yasmine-tani/retinanalysis` — a GitHub fork of the lab's original `DRezeanu/retinanalysis` repo, set up so the lab has its own copy to push to directly while still being able to pull in Dragos's future upstream updates. If you're reading this to set up your own machine, make sure `yasmine-tani/retinanalysis` is actually up to date first (push any pending local work: `git add`, `git commit`, `git push`) so a fresh clone picks up everything.

Everything you'll install, at a glance:

Tool	Why you need it	Windows	Mac
Git	Clone the repo (and its submodule)	Git for Windows	Xcode Command Line Tools or `brew install git`
Docker Desktop	Runs the local MySQL/DataJoint database	Requires WSL2	Get the right build for your chip
A C/C++ compiler	Needed to build the `vision-utils` submodule (Cython + pybind11 extensions)	Visual Studio Build Tools	Xcode Command Line Tools
Python 3.10–3.12	Runs `retinanalysis` itself	via conda or `uv`	via conda or `uv`
conda **or** `uv`	Creates the Python environment and installs packages	pick one	pick one

Everything below assumes a fresh machine with none of this installed yet — skip anything you already have.

Step 1 — Install Git

Mac: Git ships with the Xcode Command Line Tools. Open Terminal and run:

```
xcode-select --install
```

This also installs `clang`, the C/C++ compiler needed in Step 3, so it does double duty — install it now.

Windows: Download and run the installer from https://git-scm.com/download/win. Accept the defaults. This also installs **Git Bash**, a bash-compatible terminal — you'll want it in Step 5, since the repo's installer script (`install.sh`) is a bash script and won't run in plain PowerShell or Command Prompt.

Step 2 — Install Docker Desktop

The local database (MySQL, via DataJoint) runs in a Docker container, so Docker Desktop is required for any database-backed workflow (queries, `ra.populate_database()`, etc.). `import retinanalysis` itself does **not** need Docker running.

Download from https://docs.docker.com/desktop/.

Windows:

- Requires Windows 10 64-bit (version 21H2+) or Windows 11, with hardware virtualization enabled in the BIOS/UEFI, and an admin account.
- Docker Desktop uses the WSL2 backend. The installer will offer to enable and install WSL2 for you if it isn't already set up — accept that, then restart when prompted.

Mac:

- Download the build that matches your chip — Apple Silicon (M1/M2/M3/M4) or Intel. The wrong build will refuse to launch.
- Supported on macOS Ventura (13) through the current release.

Both platforms: plan for roughly 6 GB of free disk space (Docker Desktop itself plus the Linux VM/image it manages), 4 GB RAM minimum (8 GB+ recommended).

After installing, **launch Docker Desktop at least once** and leave it running whenever you need database access — you'll come back to it in Step 7.

Step 3 — Install a C/C++ Compiler

The `vision-utils` package (a required submodule, used to read Vision's `.ei`/`.bin` recording files) compiles Cython and pybind11 C++ extensions at install time. This needs an actual C/C++ compiler present on the machine — pure `pip install` will fail without one.

Mac: Already covered in Step 1 (`xcode-select --install` installs `clang`). Nothing further needed.

Windows: Install **Microsoft C++ Build Tools**:

1. Download the Visual Studio Build Tools installer from https://visualstudio.microsoft.com/visual-cpp-build-tools/.
2. In the installer, select the **"Desktop development with C++"** workload.
3. Install (this is a few GB) and restart if prompted.

You do not need full Visual Studio or an IDE — the Build Tools package alone is enough.

Step 4 — Install a Python Environment Manager

`retinanalysis` requires Python 3.10–3.12 (the repo's installer defaults to 3.11.13). Pick **either** conda or `uv` — you don't need both. If you already have Anaconda or Miniconda installed from other work, just use that; there's no need to install anything new for this step.

Option A — conda (Anaconda or Miniconda)

Anaconda and Miniconda both give you the `conda` command and work identically for this repo — Anaconda is the full distribution (bundled with lots of extra packages, larger download), Miniconda is the minimal installer (just conda + Python). If you already have either one, skip straight to Step 5.

Otherwise, download the Miniconda installer for your OS/chip from https://docs.conda.io/en/latest/miniconda.html and run it, accepting the defaults. Confirm with `conda --version` in a new terminal.

Option B — `uv`

A newer, faster alternative to conda for creating Python environments and installing packages — same basic job, just quicker, especially for installs. Not necessary if conda already works for you; only worth installing if you specifically want to try it.

Mac/Linux, in Terminal:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows, in PowerShell:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Open a **new** terminal window afterward so it picks up the updated PATH, then confirm with `uv --version`.

Step 5 — Clone and Install RetinAnalysis

Use a terminal that has `git` on its PATH — on Windows, use **Git Bash** (installed in Step 1) so the bash installer script works.

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
```

`--recursive` is required — it pulls in the `artificial-retina-software-pipeline` submodule (`lib/`) that `vision-utils` lives in. If you forget it, fix it with:

```
git submodule update --init --recursive
```

Then run the installer with whichever tool you picked in Step 4:

```
./install.sh conda
# or:
./install.sh uv
```

This creates the Python environment, installs `retinanalysis` itself (editable), builds and installs the `vision-utils` submodule package, and creates a placeholder `src/retinanalysis/config/config.ini` if one doesn't already exist. Useful variants:

```
./install.sh conda --env my_env    # custom conda environment name
./install.sh uv --dev              # also installs pytest
./install.sh uv --python 3.11.13   # pin a specific Python version
```

If Git Bash / a bash shell genuinely isn't available on Windows, you can run the same steps `install.sh` performs by hand, from PowerShell, inside your activated environment:

```
pip install -e .
pip install .\lib\artificial-retina-software-pipeline\utilities\
```

(swap `pip` for `uv pip` if using a `uv` venv). You'll then need to create `src\retinanalysis\config\config.ini` yourself — see Step 6.

Step 6 — Fix a Known Windows Matplotlib Bug

On Windows, the first `import matplotlib` (which happens indirectly via `import retinanalysis`) can raise a DLL load error. If you hit this, run:

```
pip uninstall Pillow
pip install -U Pillow
```

(`uv pip uninstall Pillow` / `uv pip install -U Pillow` if using a `uv` environment.)

Step 7 — Configure `config.ini`

`install.sh` creates a placeholder at `src/retinanalysis/config/config.ini` (or create it yourself per Step 5's manual path). Edit it with real data paths for **this** machine — it's gitignored, so every machine keeps its own copy and it's never overwritten by `git pull`.

The package auto-selects the `WINDOWS_*` / `LINUX_*` / plain (macOS, `DEFAULT`/ `SECONDARY`) sections based on the OS it's running on, so you only need to fill in whichever section(s) this machine will actually use — not all of them.

Below is Yasmine's actual working Windows config, as a real example of what a filled-in section looks like (not a placeholder). Everyone on the same shared network drive will have a very similar structure — the folder layout under the drive (`Array-data\sorted`, `Array-data\raw`, etc.) is the same for everyone, so in practice you're mainly changing the drive letter/mount point and `user` to match your own machine:

```
[WINDOWS_DEFAULT]
analysis = B:\Array-data\sorted
data = B:\Array-data\sorted
raw = B:\Array-data\raw
h5 = B:\Array-data\h5
meta = B:\Array-data\dj_meta
tags = B:\Array-data\tags
query = B:\Array-data\sorted
user = yasminetani

[WINDOWS_SECONDARY]
analysis = B:\Array-data\sorted
data = B:\Array-data\sorted
raw = B:\Array-data\raw
h5 = B:\Array-data\h5
meta = B:\Array-data\dj_meta
tags = B:\Array-data\tags
query = B:\Array-data\sorted
user = yasminetani
```

Notes for adapting this to your own machine:

- `B:` is whatever drive letter *this* Windows machine happens to have the shared network drive mapped to — yours may well be mapped to a different letter. Check what your own mapped drive is called (File Explorer → This PC) and substitute that instead of `B:`.
- `user` should be your own username, not `yasminetani`.
- `DEFAULT`/`SECONDARY` vs. `WINDOWS_DEFAULT`/`WINDOWS_SECONDARY` vs. `LINUX_DEFAULT`/`LINUX_SECONDARY` — same field names, just under the section that matches your OS. On **Mac**, the same network share is normally reached under `/Volumes/<share-name>/...` instead of a drive letter (check Finder → Go → Network, or however your lab's share is mounted) — for example:

```
[DEFAULT]
analysis = /Volumes/Array-data/sorted
data = /Volumes/Array-data/sorted
raw = /Volumes/Array-data/raw
h5 = /Volumes/Array-data/h5
meta = /Volumes/Array-data/dj_meta
tags = /Volumes/Array-data/tags
query = /Volumes/Array-data/sorted
user = your_username
```

  (the exact mount name may not literally be `Array-data` on your Mac — confirm what it's actually called once the drive is mounted, and adjust).
- `SECONDARY` is only needed if you actually have a second data location (e.g. a local SSD copy in addition to the network drive) — if not, you can leave that section out entirely, or just duplicate `DEFAULT`'s values like the example above does.
- The `query` path specifically is used for plotting mosaics/cell typing across the whole dataset archive (e.g. the NAS/network drive), even when your other paths point at a local SSD copy — keep it pointed at the shared drive.

Step 8 — Start the Local Database

Retinanalysis stores experiment metadata in a local DataJoint/MySQL database, run via Docker.

```
mkdir -p ../retinanalysis-database
cp docker-compose.yaml ../retinanalysis-database/
cd ../retinanalysis-database
docker compose up -d
```

(Older Docker installs may need the hyphenated `docker-compose up -d` instead.)

Make sure **Docker Desktop is open and the container is running** before any database-backed call — in Docker Desktop's UI you'll see a stop icon next to the running container; click the play button if it isn't running.

Step 9 — Launch Jupyter and Populate the Database

`retinanalysis` is built around Jupyter notebooks — every demo in `demos/` is one, and a notebook is the easiest way to actually run Python against this package (rather than a bare script or terminal

REPL), so this is the recommended way to start.

Install Jupyter directly **into the same environment** `install.sh` just created, one time only:

```
conda activate retinanalysis        # conda users
pip install jupyterlab              # (or: uv pip install jupyterlab, for uv users)
```

Because Jupyter lives in that same environment (rather than a separate/global Jupyter install), there's no separate kernel-registration step needed — activate the environment and launch Jupyter, and it's automatically using the right Python with `retinanalysis` already installed:

```
conda activate retinanalysis   # if not already active
jupyter lab
```

This opens a browser tab. Open any notebook in `demos/` (start with `1_retinanalysis_intro.ipynb`) and run this in a cell to populate the database:

```
import retinanalysis as ra
ra.populate_database()
```

If `config.ini` is set up correctly, this needs no arguments. `import retinanalysis` itself works without the database running — only queries and `populate_database()` actually need the container up.

Going forward, once this one-time setup is done, picking up future changes is just:

```
git pull
```

(followed by rerunning `./install.sh` only if dependencies themselves changed — a plain code/notebook update doesn't need a reinstall.)

(If you'd rather not use Jupyter at all, the same two lines work in a plain `python` interpreter or script from the activated environment — Jupyter just makes it much easier to explore results, plots, and the existing demo notebooks.)

Optional — `datajoint.json`

DataJoint 2 will print a harmless warning if it can't find a `datajoint.json` in your project root. Retinanalysis has working fallback defaults built in, so this is optional. To silence the warning explicitly, create `datajoint.json` in the repo root:

```
{
  "database.host": "127.0.0.1",
  "database.port": 3306,
  "database.user": "root",
  "database.password": "simple"
}
```

Quick Reference — Full Command Sequence

Once every prerequisite above is installed, a fresh setup is just:

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
./install.sh conda             # or: ./install.sh uv
# edit src/retinanalysis/config/config.ini with real paths
mkdir -p ../retinanalysis-database
cp docker-compose.yaml ../retinanalysis-database/
cd ../retinanalysis-database
docker compose up -d
```

```
conda activate retinanalysis
pip install jupyterlab   # one-time
jupyter lab
```

Then, in a notebook cell:

```
import retinanalysis as ra
ra.populate_database()
```

From then on, picking up future updates is just `git pull` (plus rerunning `./install.sh` only if dependencies themselves changed).

Troubleshooting

"DLL load failed" on `import matplotlib` (Windows) — see Step 6 (reinstall Pillow).

`install.sh` fails with "required submodule package is missing" — you cloned without `--recursive`. Run `git submodule update --init --recursive` from the repo root, then rerun `install.sh`.

Docker Desktop won't start / WSL2 errors (Windows) — confirm hardware virtualization is enabled in BIOS/UEFI, and that WSL2 itself is installed (`wsl --status` in PowerShell). Docker Desktop's installer can install WSL2 for you if it's missing.

Database calls fail / connection refused — Docker Desktop isn't running, or the `retinanalysis-database` container isn't started. Open Docker Desktop and confirm the container shows as running (stop icon, not play icon), or run `docker compose up -d` again from the `retinanalysis-database` directory.

Using an existing pre-DataJoint-2 database — don't try to reuse it. Create a fresh container per Step 8 and repopulate per Step 9 after updating `retinanalysis` — mixing an old DataJoint 0.14 database with the current `datajoint==2.2.2` dependency isn't supported. Keep the old environment/database around until the new one is confirmed working.

pip/uv install of `vision-utils` fails with a compiler error — the C/C++ compiler from Step 3 either isn't installed or isn't on PATH. On Windows, make sure you installed the "Desktop development with C++" workload, not just the base Build Tools installer, and try again from a fresh terminal.

Sources

- Docker Desktop system requirements: https://docs.docker.com/desktop/
- `uv` installation: https://docs.astral.sh/uv/getting-started/installation/
- Miniconda: https://docs.conda.io/en/latest/miniconda.html
- Git for Windows: https://git-scm.com/download/win
- Microsoft C++ Build Tools: https://visualstudio.microsoft.com/visual-cpp-build-tools/